AXI-Lite FIR DSP Peripheral

Modular VHDL IP core implementing a register-controlled
FIR DSP interface with scalable architecture.

Verification results

Author: Gabriela Cabrera
Github: @MGabrielaCabrera
Date: March 2026
Version: 1.0

INDEX

1. Verification description

This document presents the results of the initial verification phase for the VHDL modules developed in this project. The primary objective of this phase is to perform a first functional validation of each module individually through simulation.

The verification has been carried out using VHDL testbenches that rely on assertion-based checks to validate the expected behavior of the design. These assertions allow the simulation to automatically detect functional mismatches and ensure that the implemented logic behaves according to the intended specification.

For this first verification stage, each module has been tested independently. The document includes simulation waveform captures and relevant observations for each test case in order to illustrate the correct operation of the modules under different conditions.

It should be noted that this verification stage focuses on basic functional validation. In a later phase of the project, the verification environment will be extended using more advanced methodologies, including SystemVerilog-based testbenches, functional coverage, and a more structured verification framework. These improvements will enable a more comprehensive validation of the design and increase confidence in the correctness and robustness of the implementation.

2. AXI-Lite Slave Interface Verification

Module Description

The AXI-Lite Slave Interface module implements a memory-mapped interface compliant with the AXI4-Lite protocol. Its main function is to translate AXI-Lite bus transactions into a simplified internal register interface used by the rest of the design.

The module handles both read and write transactions through the standard AXI-Lite channels: write address, write data, write response, read address, and read data.

Additionally, the module performs basic protocol checks, including address alignment verification and address range validation. When an invalid or misaligned address is detected, the module returns an AXI-Lite slave error response (SLVERR).

Testbench Architecture

The testbench instantiates the design under test (DUT) and emulates the behavior of an AXI-Lite master.

A clock generator provides the system clock with a period of 10 ns, while a reset sequence initializes the DUT before the start of the tests. The testbench includes a simple register file model composed of six 32-bit registers representing the memory-mapped registers accessible through the AXI-Lite interface.

The register file reacts to the internal control signals generated by the DUT:

- *reg_write_en* triggers register updates.
- *reg_read_en* enables register read operations.

Two procedures are implemented to simplify the stimulus generation:

- *axi_write:* performs a complete AXI-Lite write transaction including address, data, and response handshake.
- *axi_read:* performs a complete AXI-Lite read transaction including address and read data handshake.

These procedures allow the testbench to generate multiple test scenarios while respecting the AXI-Lite handshake protocol.

Test Scenarios

A set of directed tests has been implemented using assertion-based verification. Each test stimulates a specific aspect of the AXI-Lite protocol behavior and checks the correctness of the module using VHDL assertions.

The following test scenarios are executed:

1. **Basic Write Transaction**: A write operation is performed to register 0. The test verifies that the write response is OKAY and that the register file is updated with the expected value.
2. **Write to Another Valid Registe**r: A write operation is issued to register 1 to confirm correct address decoding and register update behavior.
3. **Read Transaction Verification:** The value previously written to register 0 is read back and compared with the expected data.
4. **Read from Another Register:** A read operation is performed on register 1 to verify correct readback functionality.
5. **Partial Write Using Byte Strobes:** A write transaction using partial byte enables (`WSTRB`) verifies that only selected bytes of the register are updated.
6. **Misaligned Write Address:** A write transaction is attempted using a misaligned address. The expected response is an AXI-Lite slave error (`SLVERR`).

7. **Out-of-Range Write Address:** A write is issued to an address outside the valid register range. The module should return a `SLVERR` response.
8. **Misaligned Read Address:** A read transaction using a misaligned address verifies that the module detects alignment errors.
9. **Out-of-Range Read Address:** A read operation targeting an invalid address verifies correct error handling.
10. **Back-to-Back Write Transactions:** Multiple consecutive writes are issued to different registers to verify correct operation under continuous traffic.
11. **Write Followed by Immediate Read:** A write operation is immediately followed by a read to confirm that the updated data can be retrieved correctly.

Simulation Results

The simulation results confirm the correct functional behavior of the AXI-Lite Slave Interface module for all tested scenarios.

All transactions follow the expected AXI-Lite handshake protocol. Valid write transactions correctly update the internal register file, and subsequent read operations return the expected data values. The byte strobe functionality correctly enables partial register updates according to the specified write mask.

Error handling mechanisms were also verified. Misaligned and out-of-range addresses correctly trigger AXI-Lite slave error responses (`SLVERR`), demonstrating that the implemented address validation logic functions as intended.

The accompanying waveform captures illustrate representative transactions, including valid read and write operations, byte strobe behavior, and error responses for invalid accesses.

```
Analyzing design files
Elaborating axi_lite_slave_if testbench
Running axi_lite_slave_if_tb testbench. Waveforms saved to waves/waves_axi_lite_slave_if_tb.fst
axi_lite_slave_if_tb.vhd:231:9:@50ns:(report note): Test 1: Write to register 0 and verify response
axi_lite_slave_if_tb.vhd:241:9:@90ns:(report note): Test 2: Write to register 1 and verify response
axi_lite_slave_if_tb.vhd:251:9:@140ns:(report note): Test 3: Read back register 0 and verify data
axi_lite_slave_if_tb.vhd:262:9:@220ns:(report note): Test 4: Read back register 1 and verify data
axi_lite_slave_if_tb.vhd:275:9:@300ns:(report note): Test 5: Write with partial byte strobes (upper two bytes only)
axi_lite_slave_if_tb.vhd:285:9:@400ns:(report note): Test 6: Write to misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:293:9:@450ns:(report note): Test 7: Write to out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:301:9:@500ns:(report note): Test 8: Read from misaligned address -> expect SLVERR
axi_lite_slave_if_tb.vhd:309:9:@580ns:(report note): Test 9: Read from out-of-range address -> expect SLVERR
axi_lite_slave_if_tb.vhd:317:9:@660ns:(report note): Test 10: Back-to-back writes to consecutive registers
axi_lite_slave_if_tb.vhd:331:9:@810ns:(report note): Test 11: Write then immediate read (reg 3)
axi_lite_slave_if_tb.vhd:342:9:@990ns:(assertion note): End of simulation - all tests passed
Opening waveform viewer (only if the fst file exists)
Test complete!
```

Figure 1 – Test results

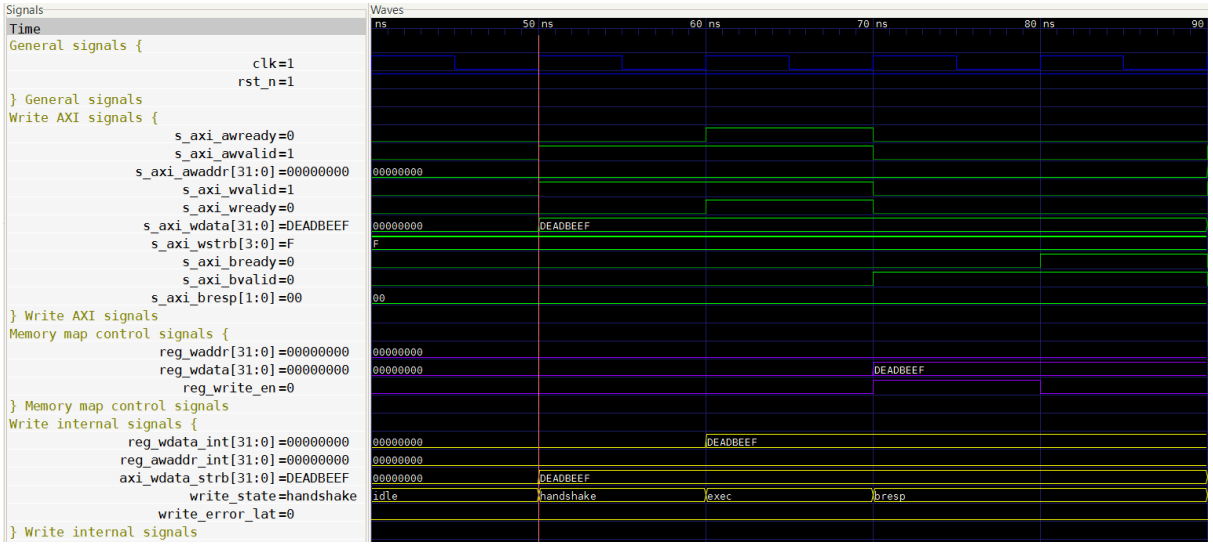

Figure 2 – Test 1: AXI-Lite Write Transaction (Register 0)

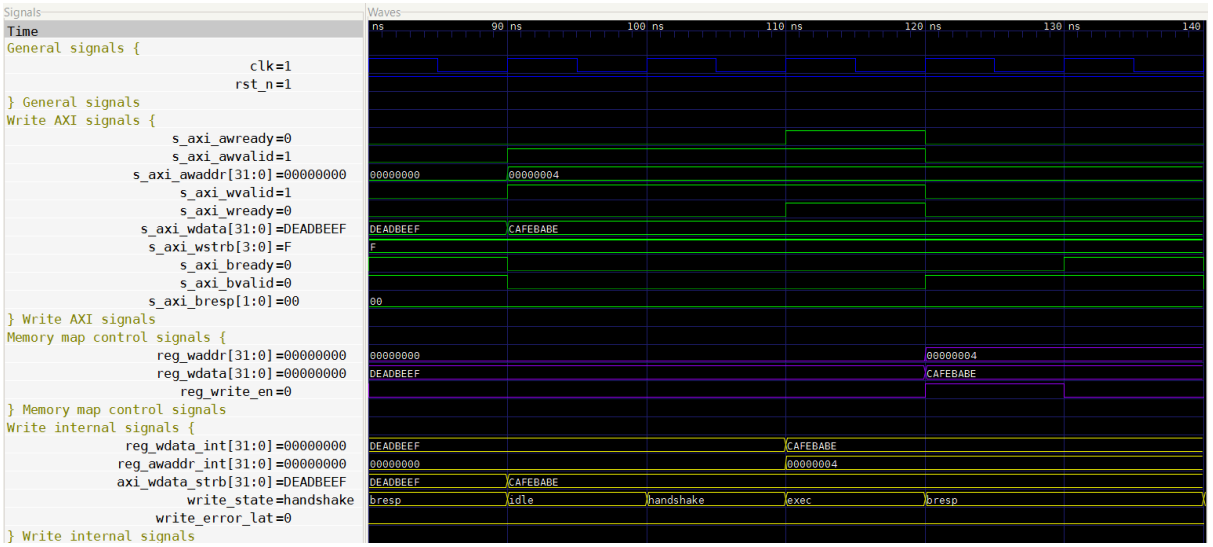

Figure 3 – Test 2: AXI-Lite Write Transaction (Register 1)

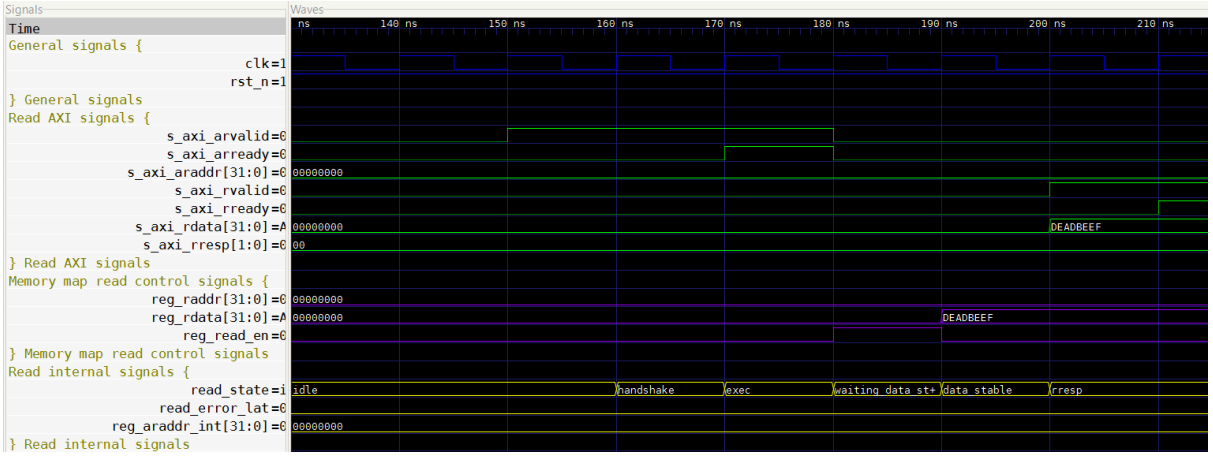

Figure 4 – Test 3: AXI-Lite Read Transaction (Register 0)

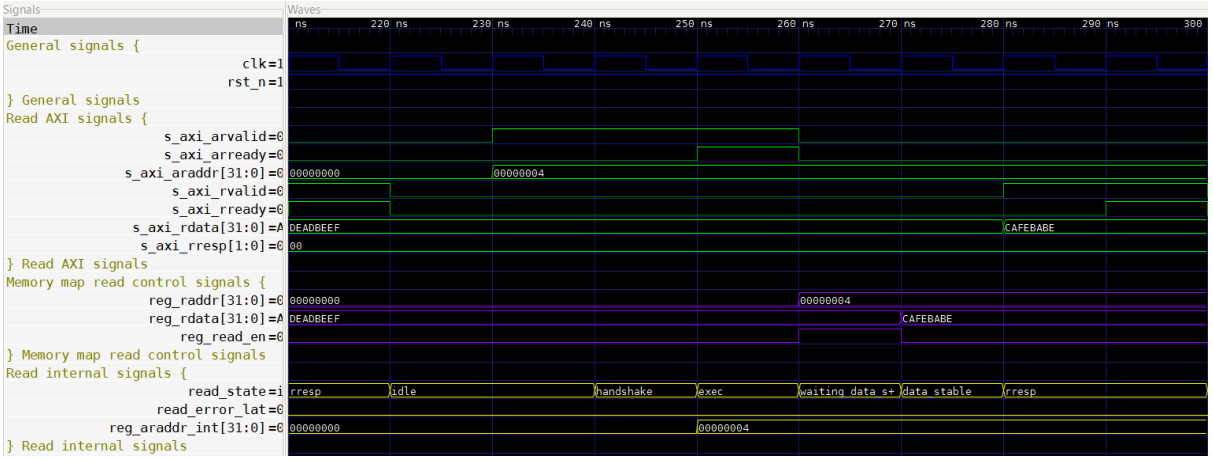

Figure 5 – Test 4: AXI-Lite Read Transaction (Register 1)

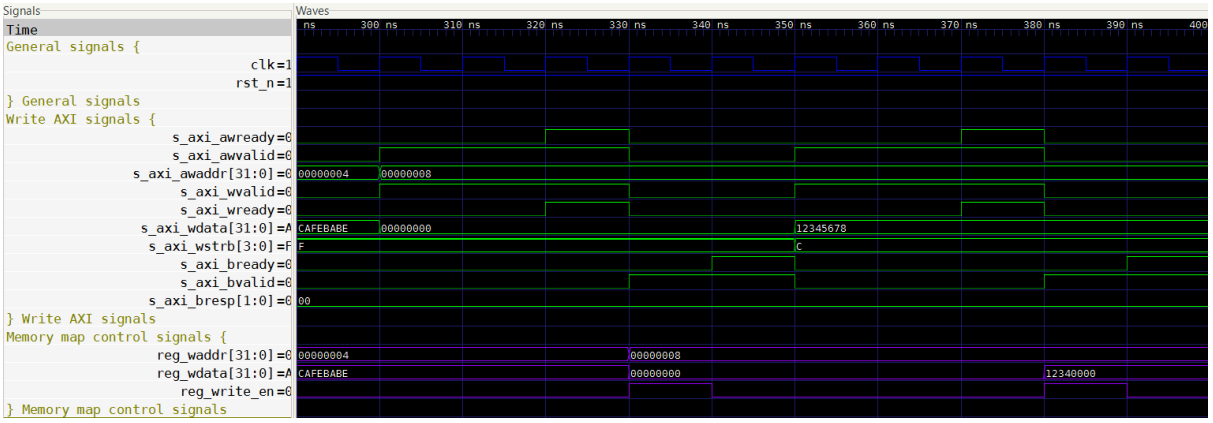

Figure 6 – Test 5: Partial Write Using Byte Strobes

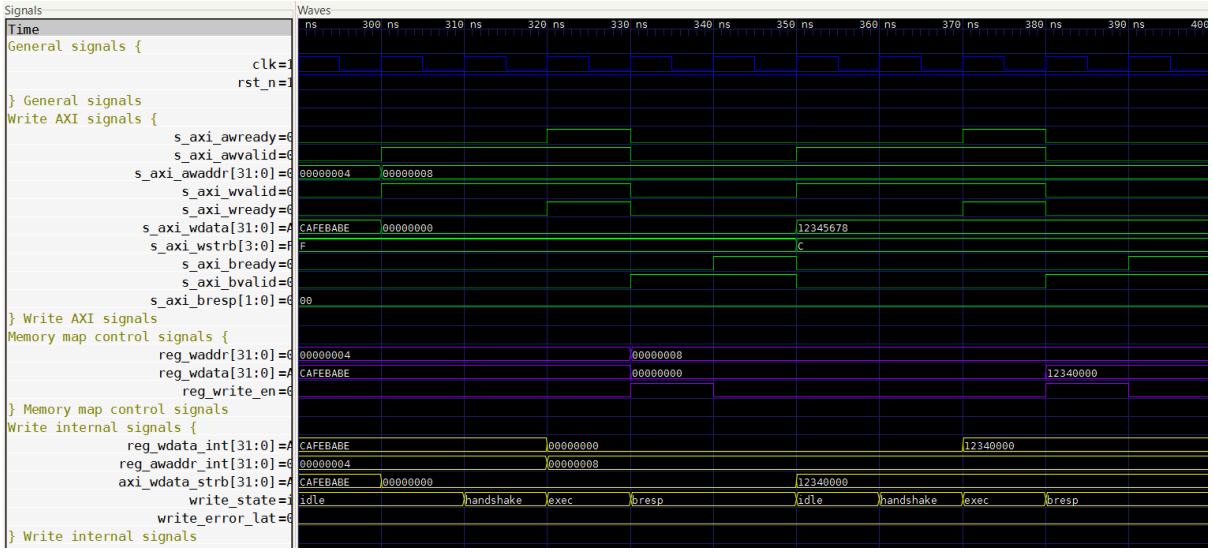

Figure 7 – Test 6: Misaligned Address Error Response

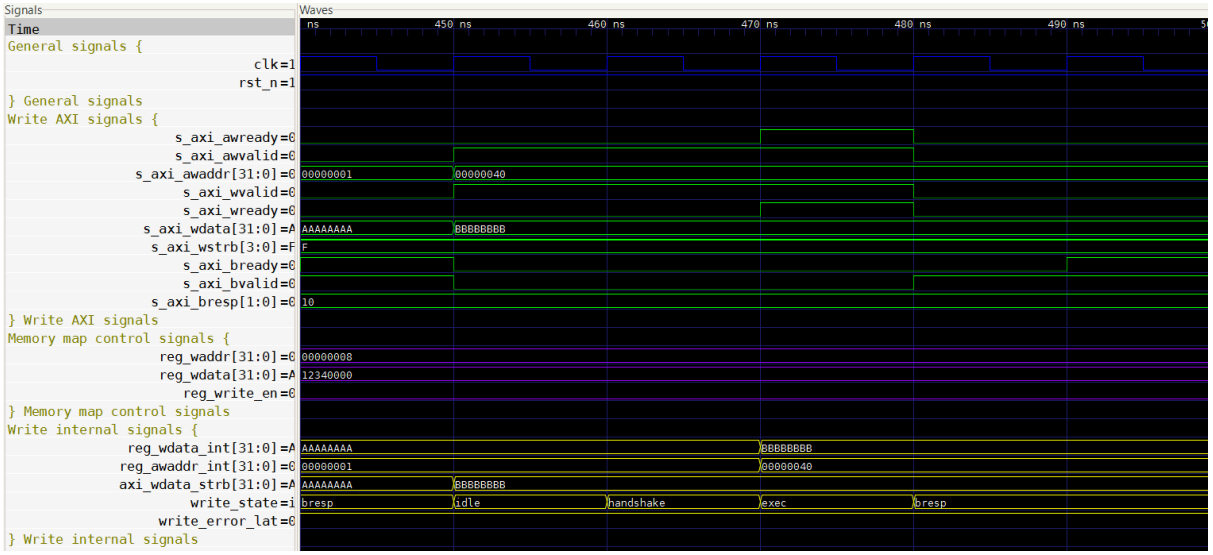

Figure 8 - Test 7: Out-of-Range Write Address

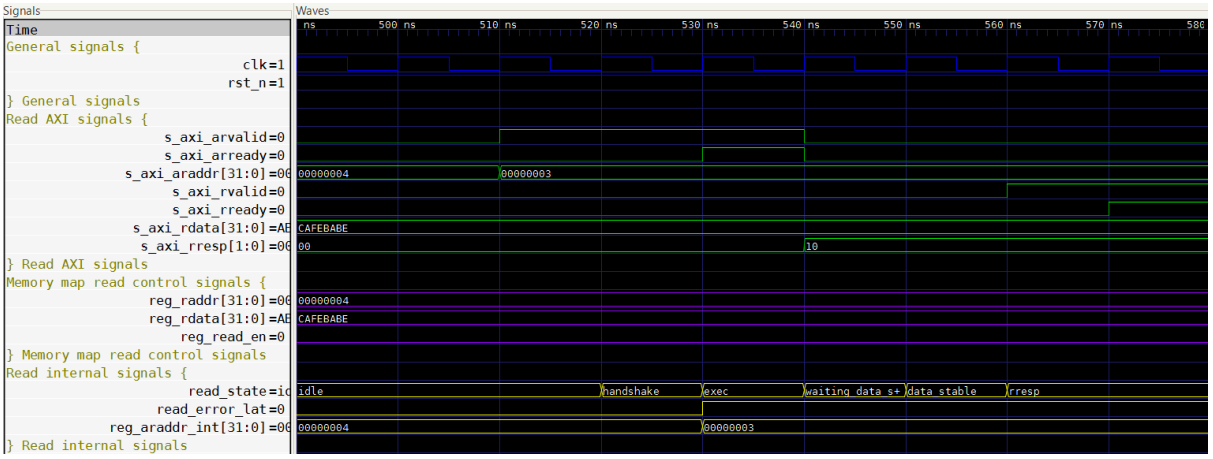

Figure 9 - Test 8: Misaligned Read Address

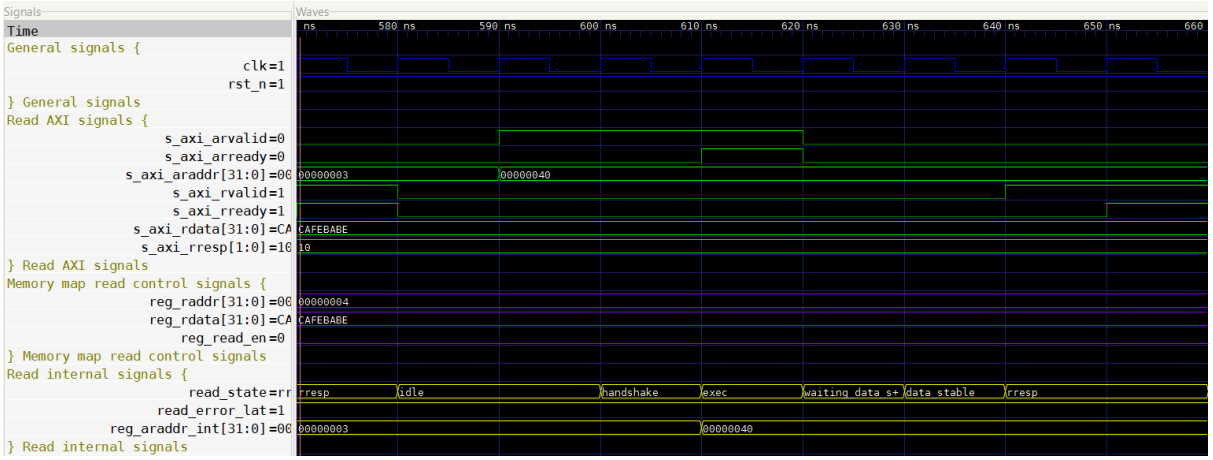

Figure 10 - Test 9: Out-of-Range Read Address

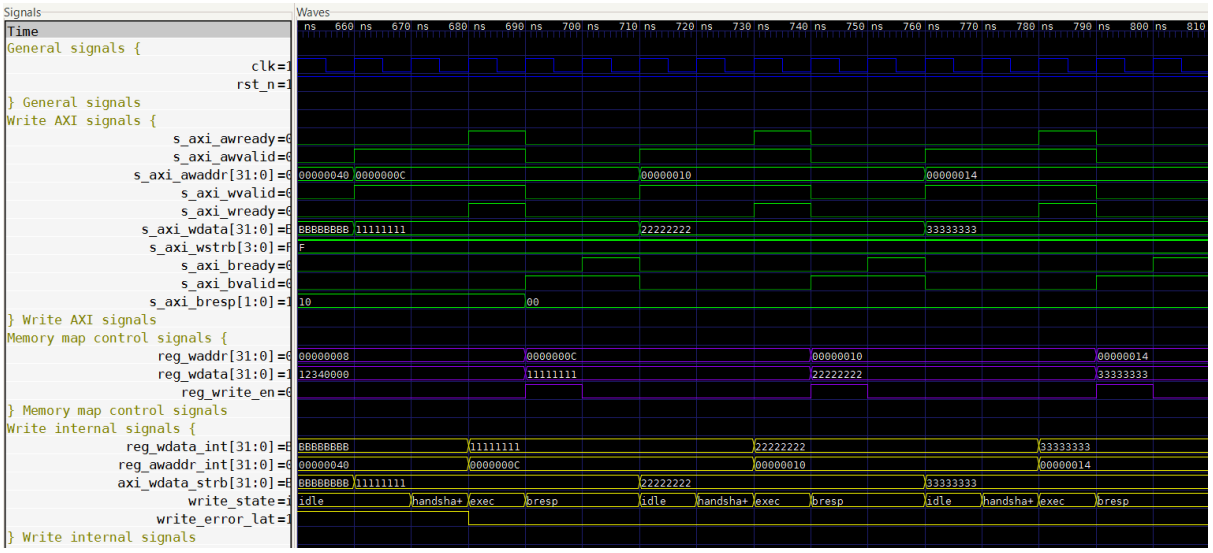

Figure 11 - Test 10: Back-to-Back Write Transactions

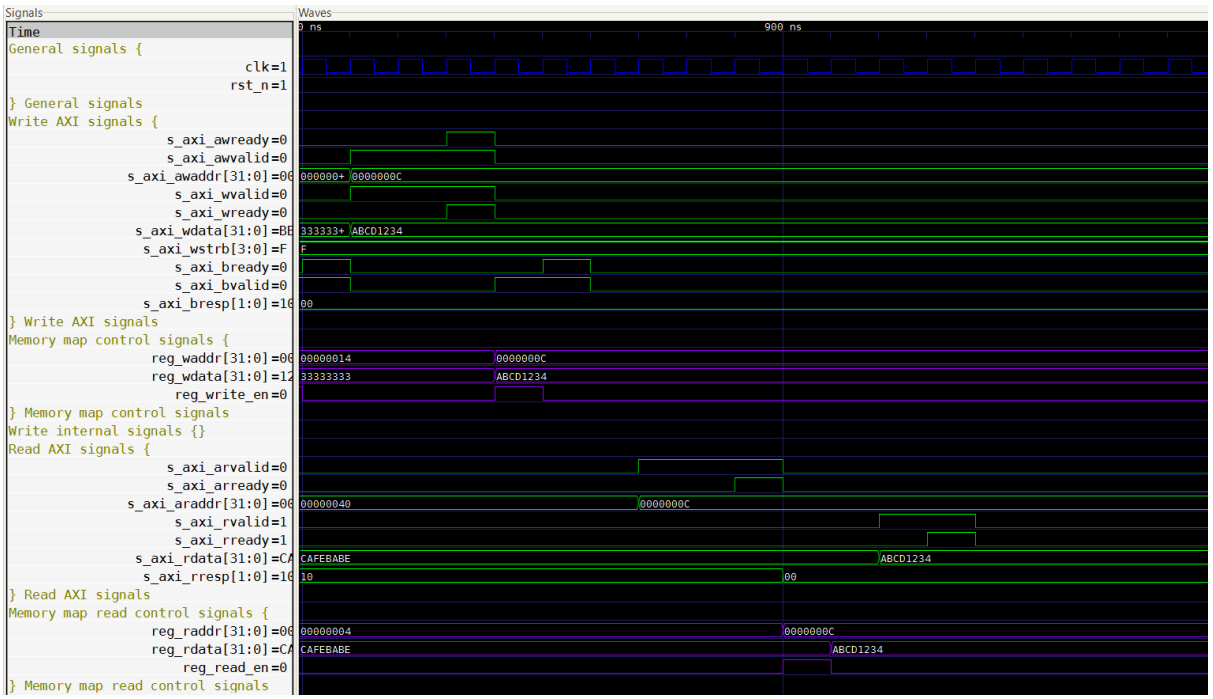

Figure 12 - Test 11: Write Followed by Immediate Read

3. Control FSM Verification

Module Description

The Control FSM module implements the control logic responsible for coordinating communication between the memory-mapped register interface and an external FIR DSP processing module.

The module operates as a finite state machine that manages two main types of transactions: coefficient loading and sample processing. Input samples and configuration data are received through a register interface and forwarded to the DSP through streaming-style handshake signals.

The FSM includes four operational states:

- **IDLE** – default state where the module waits for new commands.
- **SEND** – an input sample is sent to the DSP.
- **WAIT_RESULT** – the FSM waits for the DSP to produce a processed output sample.
- **LOAD_COEFF** – a coefficient value is written to the DSP coefficient interface.

The module also generates status information through the reg_status signal, indicating whether the module is ready, busy, or in an error condition. Control signals such as dsp_enable, dsp_reset, and dsp_mode are passed through directly from the register interface to the DSP.

The FSM ensures correct sequencing of operations, manages handshake signals for streaming data transfers, and prevents invalid operations such as sending samples when the DSP is disabled or attempting to update coefficients while the DSP is active.

Testbench Architecture

The testbench instantiates the design under test (DUT) and drives the register interface and DSP handshake signals to emulate the behavior of the surrounding system.

A clock generator produces the system clock with a period of 10 ns, while a reset sequence initializes the FSM before the execution of the verification scenarios.

The testbench provides stimulus to both sides of the interface:

- **The register interface**, which controls the module through signals such as reg_enable, reg_mode, reg_data_in, sample_write_strobe, and coeff_write_strobe.
- **The DSP interface**, which emulates the behavior of the external DSP through signals such as dsp_data_in_ready, dsp_data_out_valid, and dsp_data_out.

Two helper procedures are used to simplify test stimulus generation:

- **clk_rise** – waits for a rising clock edge.
- **clk_fall** – waits for a falling clock edge.

These procedures ensure that inputs are applied during the stable half of the clock cycle so that the DUT samples them correctly at the next rising edge.

Test Scenarios

A set of directed tests has been implemented to validate the functional behavior of the FSM under different operating conditions.

The following verification scenarios are executed:

1. **Reset State:** The module is held in reset and all outputs are verified to remain in their safe default values. This ensures the FSM initializes correctly and no unintended control signals are asserted.
2. **Passthrough Signal Verification:** The signals *dsp_enable, dsp_reset*, and *dsp_mode* are verified to correctly mirror the corresponding register values. The test confirms that these signals update on the next clock cycle regardless of the FSM state.
3. **Valid Coefficient Write**: A coefficient write request is issued while the DSP is disabled. The test verifies that the FSM transitions from IDLE to LOAD_COEFF, generates a single-cycle write enable pulse, and forwards the correct coefficient address and data.
4. **Blocked Coefficient Write**: A coefficient write request is attempted while the DSP is enabled. The FSM must reject the operation, remain in IDLE, and report an error condition.
5. **Full Data Path Transaction:** A complete data processing transaction is executed. An input sample is sent to the DSP, the FSM waits for the result, and the processed output is captured and forwarded to the register interface. Handshake signals and status updates are verified during the process.
6. **Blocked Sample Transmission:** A sample write request is issued while the DSP is disabled. The FSM must reject the operation and generate an error status without initiating a data transfer.
7. **WAIT_RESULT State Hold:** The FSM is forced into the WAIT_RESULT state and the DSP output valid signal is intentionally delayed. The test verifies that the FSM remains in the waiting state and maintains the busy status until a valid result is received.
8. **Output Ready Pulse Width:** The *dsp_data_out_ready* signal is verified to produce a single clock-cycle pulse when capturing a DSP result, even if the DSP keeps the valid signal asserted for multiple cycles.
9. **Mode Update During Active Transaction:** Changes to the reg_mode register are applied while the FSM is processing a transaction. The test verifies that the updated mode propagates correctly to the dsp_mode signal without interfering with the FSM operation.

10. **Back-to-Back Coefficient Writes**: Two consecutive coefficient write operations are issued to verify that the FSM can correctly handle sequential transactions. Each operation must generate an independent write enable pulse and the FSM must return to the IDLE state between writes.

Simulation Results

The simulation results confirm the correct functional behavior of the Control FSM module across all tested scenarios.

The FSM transitions correctly between the IDLE, SEND, WAIT_RESULT, and LOAD_COEFF states according to the applied stimuli. Streaming handshake signals between the control logic and the DSP are generated with the correct timing and remain stable until the handshake is completed.

Coefficient write operations are correctly executed only when the DSP is disabled, while invalid operations are properly rejected and reported through the status signal. The data path verification confirms that input samples are correctly forwarded to the DSP and that the resulting processed data is captured and stored in the register interface.

Additional tests verify correct handling of extended wait periods, single-cycle handshake pulses, dynamic mode updates, and consecutive coefficient writes.

The accompanying waveform captures illustrate representative scenarios including coefficient loading, sample processing transactions, waiting for DSP results, and error handling conditions.

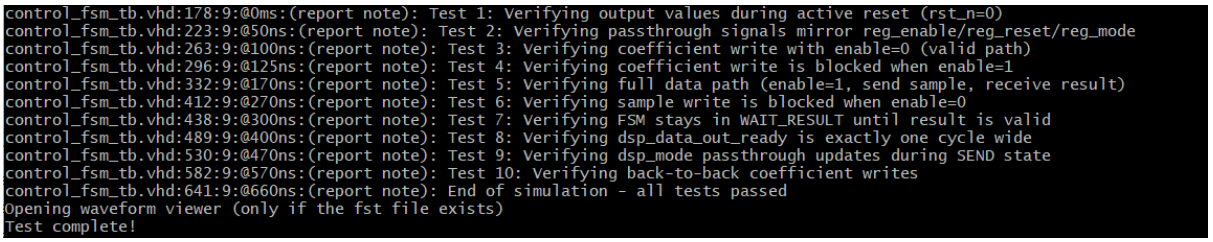

Figure 13 – Test results

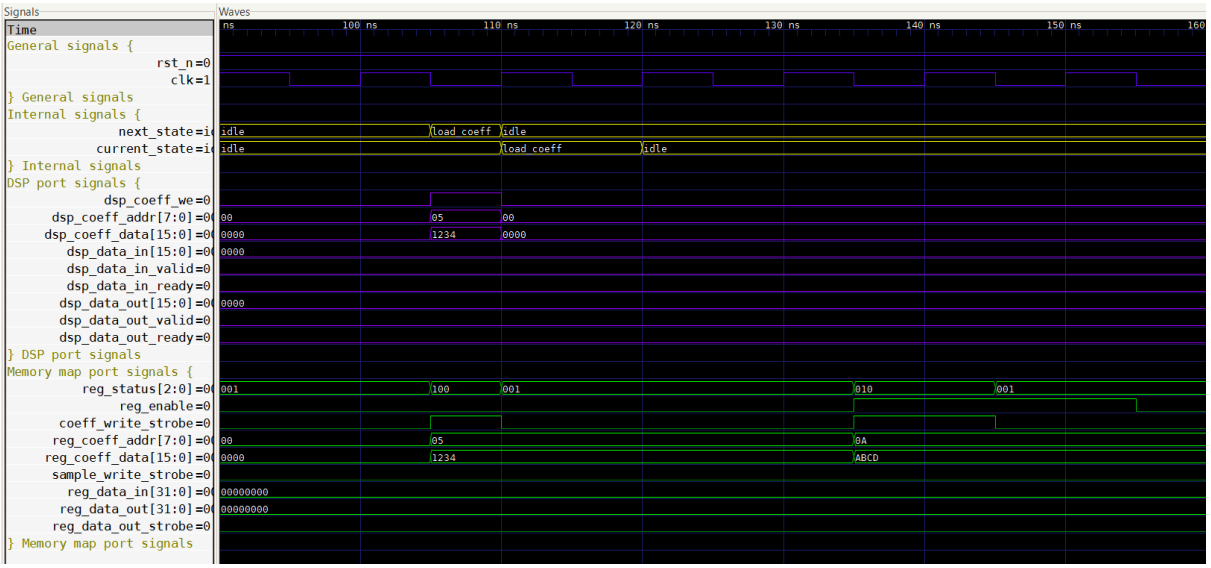

Figure 14 – Test 3 and 4

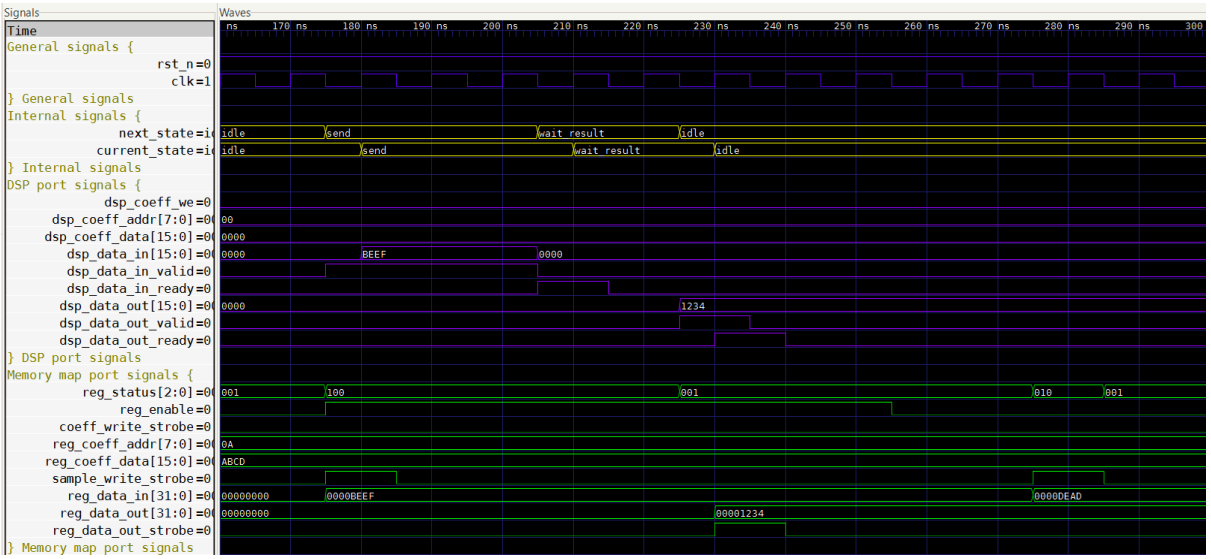

Figure 15 – Test 5 and 6

In test 5, dsp_data_in_ready must be asserted when dsp_data_in_valid is high. This is an error on the testbench.

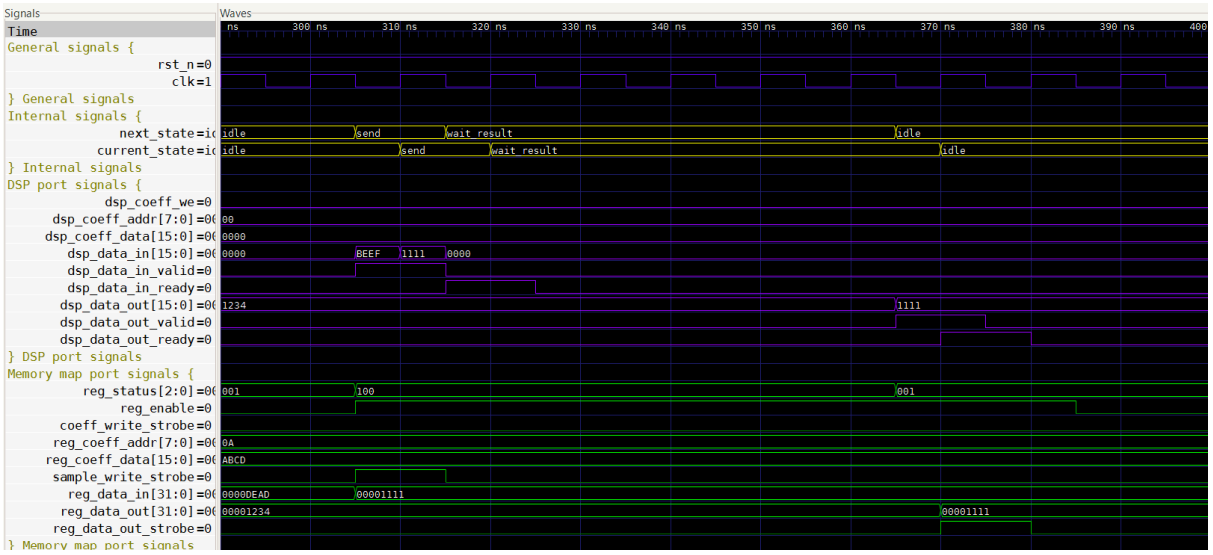

Figure 16 – Test 7

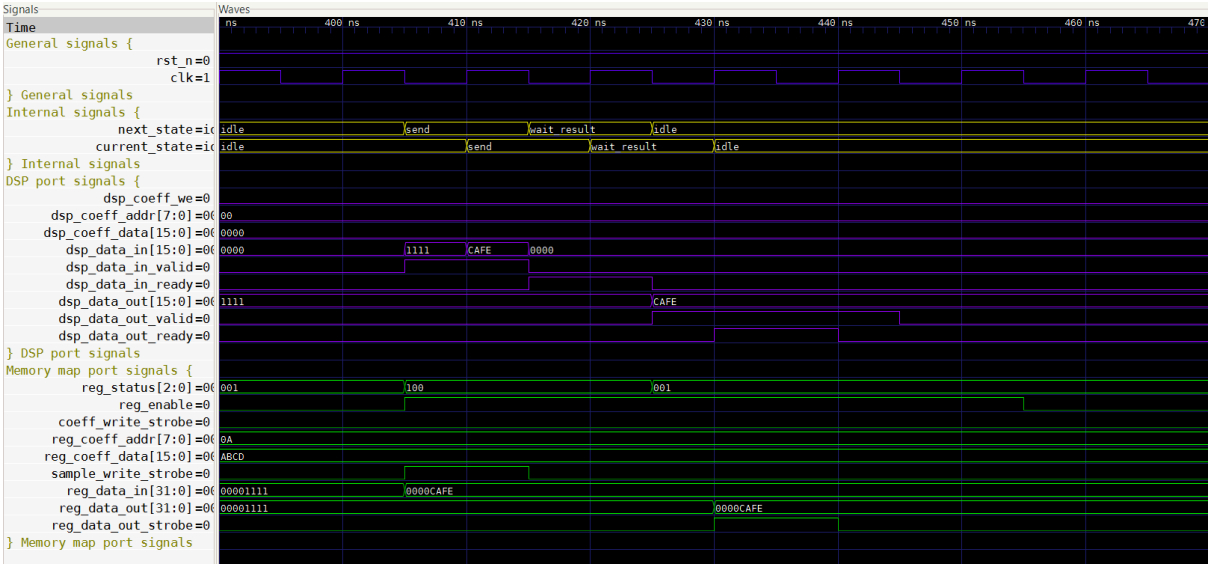

Figure 17 – Test 8

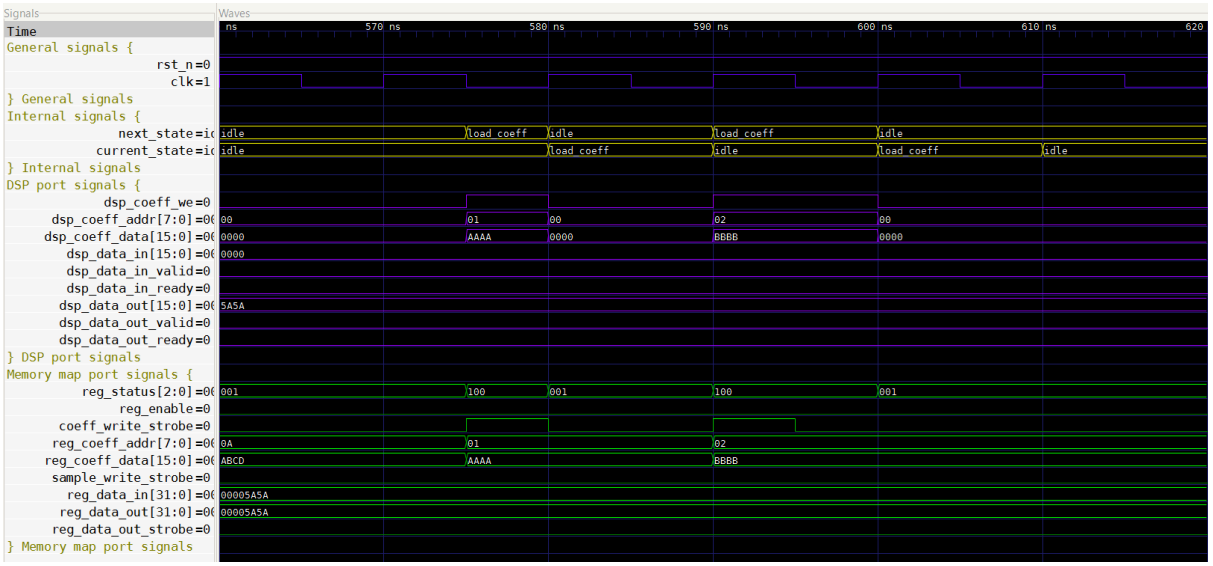

Figure 18 – Test 10

4. Register bank Verification

Module Description

The Register Bank module implements the memory-mapped register interface used to configure and monitor the DSP processing system. It acts as the bridge between the AXI-Lite slave interface and the control FSM by storing configuration parameters and forwarding control signals to the rest of the design.

The module contains several registers accessible through the internal register interface. These registers include control, status, configuration, and data registers used by the DSP control logic.

The main registers implemented in the module are:

- **CTRL register (0x00)** – Contains configuration bits controlling the DSP operation, including enable, reset, and operating mode selection.
- **STATUS register (0x04)** – Provides status information generated by the control FSM, including ready, error, and busy indicators. This register is read-only from the software perspective.
- **COEFF_DATA register (0x08)** – Stores the coefficient value that will be written to the DSP coefficient memory.
- **COEFF_ADDR register (0x0C)** – Contains the address of the DSP coefficient to be updated.
- **DATA_IN register (0x10)** – Holds the input data sample that will be sent to the DSP.
- **DATA_OUT register (0x14)** – Stores the processed data received from the DSP. This register is updated by hardware and is read-only from the software interface.

In addition to storing configuration data, the register bank generates control signals used by the control FSM. These include *sample_write_strobe* and *coeff_write_strobe*, which indicate when new data or coefficients have been written. The module also ensures that read-only registers cannot be modified through software writes and that reserved bits remain cleared.

Testbench Architecture

The testbench instantiates the design under test (DUT) and emulates the behavior of the internal register interface that would normally be driven by the AXI-Lite slave module.

A clock generator produces a system clock with a period of 10 ns, and a reset sequence initializes the DUT before the execution of the verification tests.

The testbench drives the following input signals:

- register write and read enable signals (*reg_write_en, reg_read_en*)

- register addresses *(reg_waddr, reg_raddr*)
- register write data (*reg_wdata*)
- status and data signals coming from the control FSM (*reg_status, reg_data_out, reg_data_out_strobe*)

The outputs of the register bank are monitored to verify correct behavior, including:

- configuration outputs (*reg_enable, reg_reset, reg_mode*)
- coefficient interface signals (*reg_coeff_data, reg_coeff_addr*)
- DSP input data (*reg_data_in*)
- control strobes (*sample_write_strobe, coeff_write_strobe*)
- read data (*reg_rdata*)

To simplify stimulus generation, several helper procedures are implemented in the testbench:

- **reg_write** – performs a complete register write transaction by asserting the write enable signal for a single clock cycle while providing the target address and write data.
- **reg_read** – performs a register read operation and allows the read data to be sampled from the combinational read path.

All functional checks are implemented using assertion-based verification, where the expected output behavior is validated using VHDL assertions after each stimulus sequence.

Test Scenarios

A set of directed tests has been implemented to verify the functional correctness of the register bank. Each test targets a specific aspect of the module behavior and validates the expected outputs using assertions.

The following test scenarios are executed:

1. **Reset State:** The module is held in reset to verify that all registers and outputs initialize to their safe default values. Control signals and write strobes must remain inactive during reset.
2. **CTRL Register Write and Readback:** A write operation to the CTRL register is performed to update the DSP enable, reset, and mode fields. The test verifies that the internal outputs are updated correctly and that the same values can be read back from the register.
3. **Reserved Bits Protection:** A write operation with all bits set is issued to the CTRL register. The test verifies that only the valid control bits are stored and that all reserved bits remain zero when read back.
4. **STATUS Register Read-Only Behavior:** A write attempt is issued to the STATUS register. The test verifies that the write operation is ignored and that the register contents remain controlled exclusively by the hardware status input.

5. **STATUS Register Update from FSM:** Changes applied to the reg_status input signal are verified to propagate correctly to the STATUS register and be visible through the read interface.
6. **Coefficient Data Write Operation:** A write to the COEFF_DATA register must update the coefficient value and generate a one-cycle coeff_write_strobe signal.
7. **Coefficient Address Write Operation:** Writing the COEFF_ADDR register must update the coefficient index and also generate a one-cycle coeff_write_strobe signal.
8. **Data Input Write Operation:** Writing to the DATA_IN register must update the input data value and generate a one-cycle sample_write_strobe signal that triggers sample processing in the control FSM.
9. **DATA_OUT Hardware Update:** The DATA_OUT register is updated by the control FSM through the reg_data_out_strobe signal. The test verifies that the register captures the incoming value correctly and that software write attempts are ignored.
10. **Write Priority over Simultaneous Read:** A scenario where both read and write operations occur in the same clock cycle is tested. The module must prioritize the write operation, suppress the read data during the conflict cycle, and commit the new value to the register.

Simulation Results

The simulation results confirm the correct functional behavior of the Register Bank module for all tested scenarios.

All registers correctly initialize to their defined reset values and respond appropriately to write and read operations. Writable registers update their contents according to the provided write data, while read-only registers ignore software write attempts and remain controlled exclusively by hardware signals.

The generation of control strobes for coefficient updates and sample input writes was verified to produce single-cycle pulses as expected. These signals are essential for synchronizing the register bank with the control FSM and DSP processing pipeline.

The STATUS register correctly reflects changes in the hardware status input, and the DATA_OUT register updates properly when triggered by the FSM strobe signal. Additionally, protection mechanisms for reserved bits and read-only registers were validated successfully.

The accompanying waveform captures illustrate representative test cases, including register writes, readback operations, hardware-driven register updates, and the generation of control strobes for DSP operations.

```
register_bank_tb.vhd:184:9:@0ms:(report note): Test 1: Verifying reset state
register_bank_tb.vhd:226:9:@50ns:(report note): Test 2: Verifying CTRL register write and readback
register_bank_tb.vhd:267:9:@115ns:(report note): Test 3: Verifying CTRL reserved bits are forced to zero
register_bank_tb.vhd:292:9:@175ns:(report note): Test 4: Verifying STATUS register is read-only
register_bank_tb.vhd:322:9:@235ns:(report note): Test 5: Verifying STATUS register tracks reg_status changes
register_bank_tb.vhd:366:9:@310ns:(report note): Test 6: Verifying COEFF_DATA write generates one-cycle coeff_write_strobe
register_bank_tb.vhd:395:9:@335ns:(report note): Test 7: Verifying COEFF_ADDR write generates one-cycle coeff_write_strobe
register_bank_tb.vhd:438:9:@385ns:(report note): Test 8: Verifying DATA_IN write generates one-cycle sample_write_strobe
register_bank_tb.vhd:475:9:@415ns:(report note): Test 9: Verifying DATA_OUT updates from FSM strobe and is read-only
register_bank_tb.vhd:524:9:@485ns:(report note): Test 10: Verifying write takes priority over simultaneous read
register_bank_tb.vhd:566:9:@595ns:(report note): End of simulation
```

Figure 19 – Test results

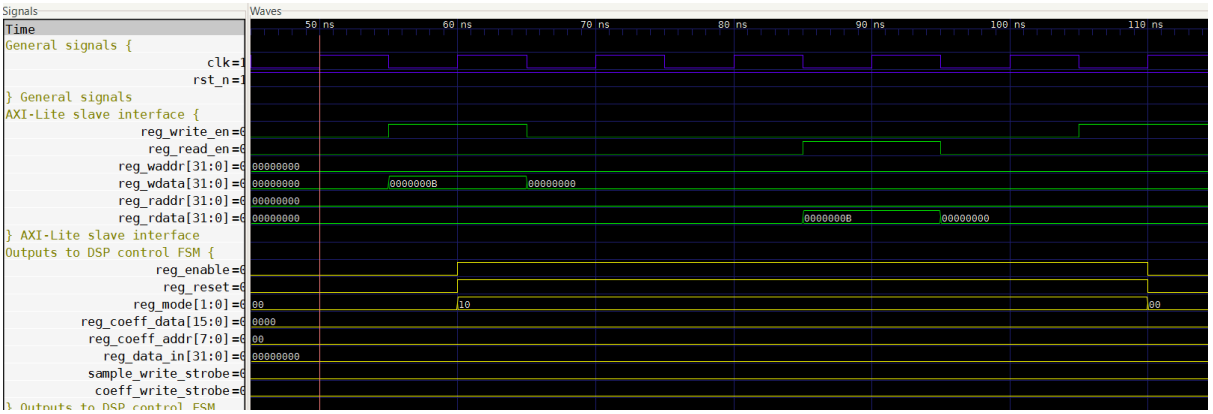

Figure 20 - Test 2

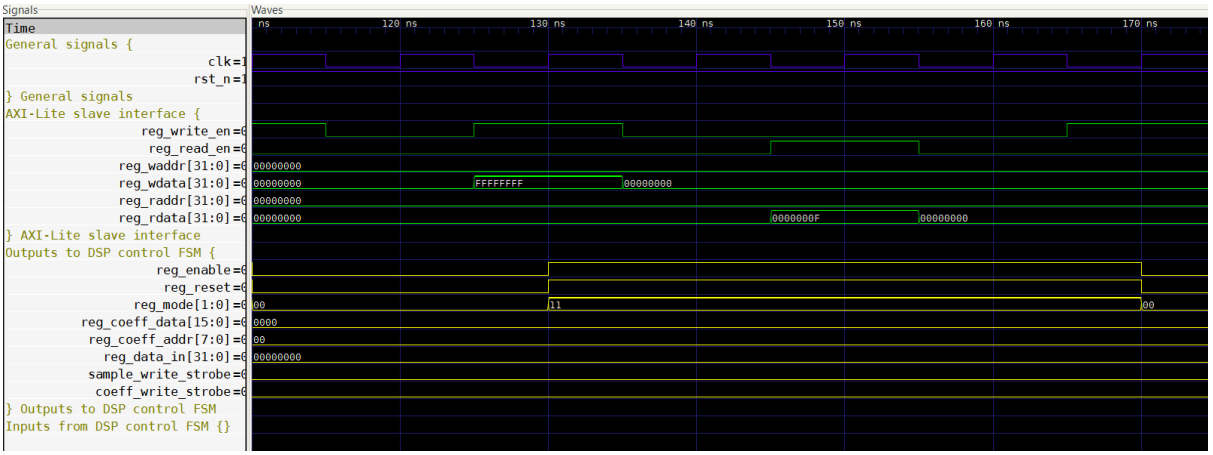

Figure 21 - Test 3

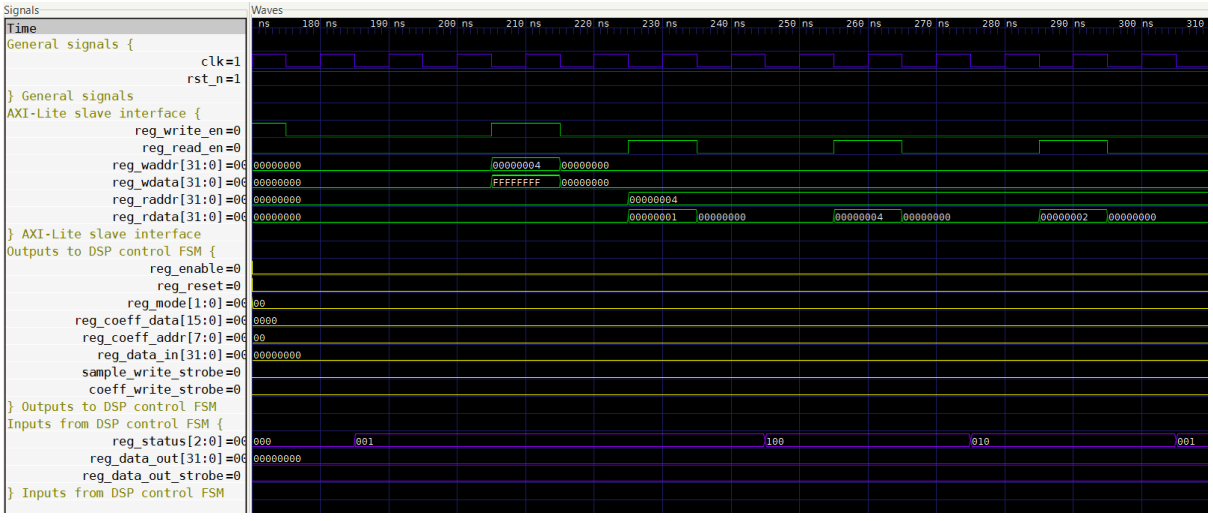

Figure 22 - Test 4 and 5

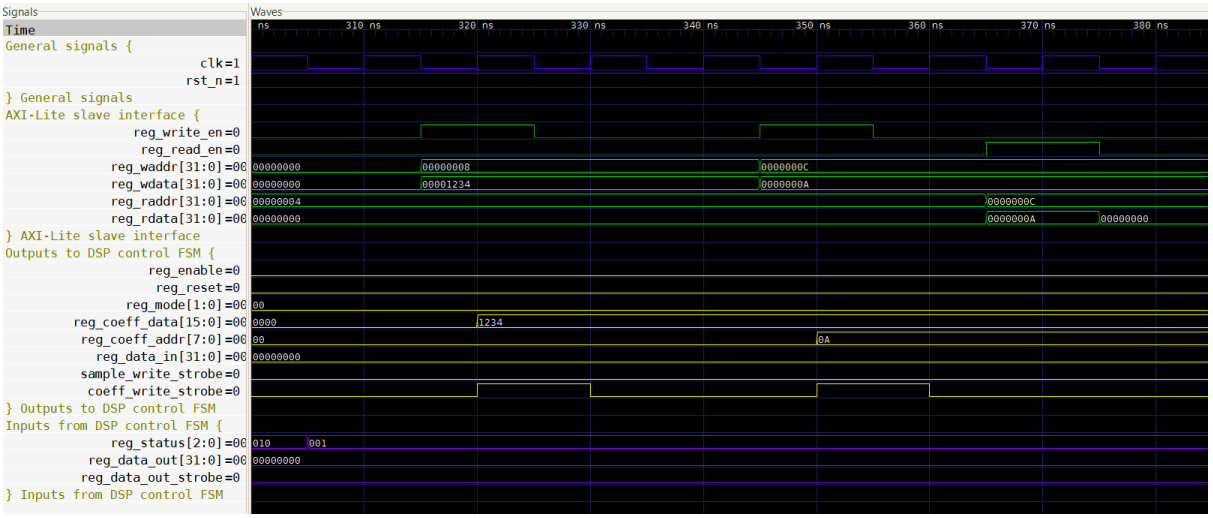

Figure 23 - Test 6 and 7

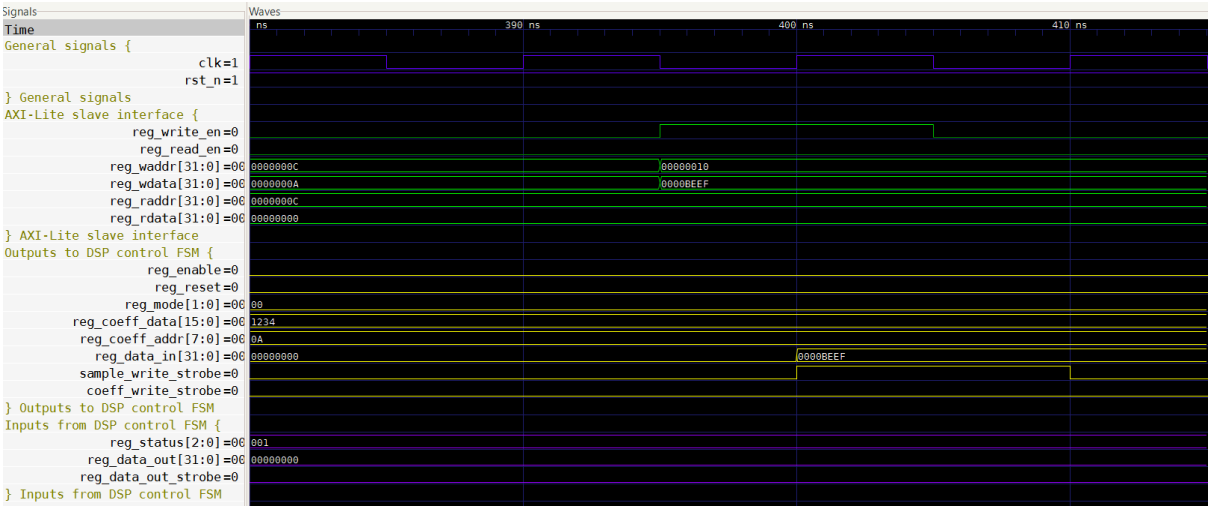

Figure 24 - Test 8

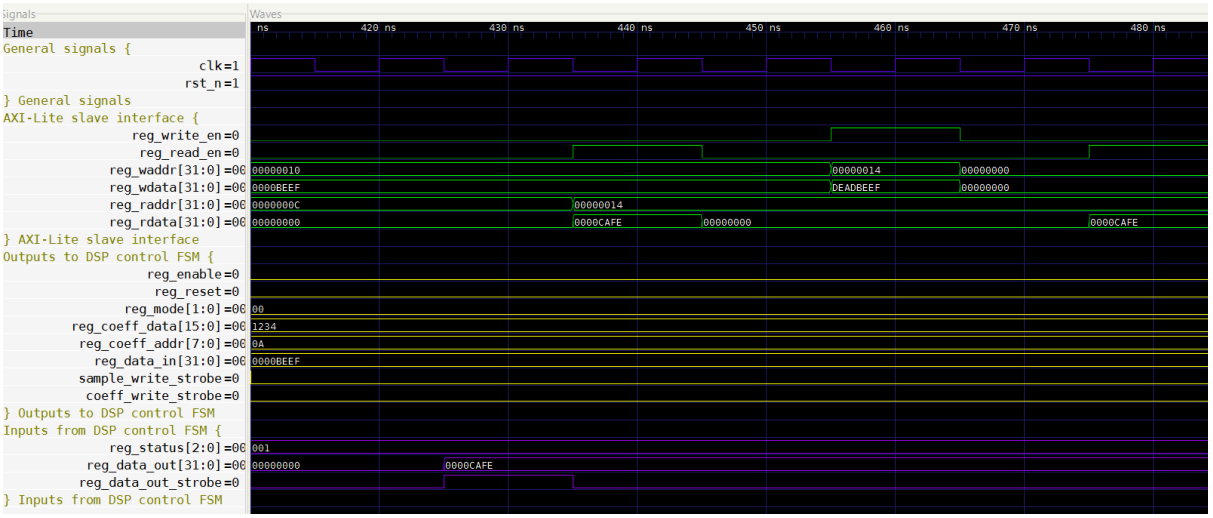

Figure 25- Test 9

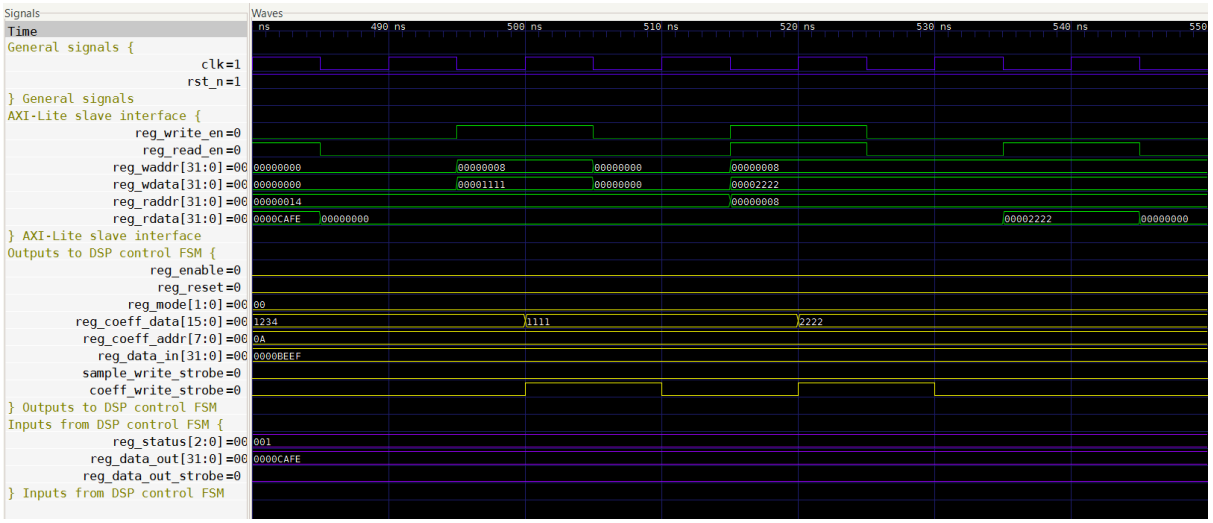

Figure 26 - Test 10

4. Top level wrapper Verification

Module Description

The Top-Level Wrapper module integrates all the main components of the design, including the AXI-Lite slave interface, the register bank, and the control FSM that manages the interaction with the DSP core.

Testbench Architecture

The testbench instantiates the DUT and drives it exclusively through the AXI-Lite interface, emulating the behavior of an external AXI master. All register accesses are performed using valid AXI transactions, ensuring that the protocol implementation is fully verified.

A clock signal with a period of 10 ns is generated, and a reset sequence initializes the system before executing the test scenarios.

Two main procedures are implemented to abstract AXI transactions:

- **axi_write** – Performs a complete AXI write transaction, including address and data phases, waiting for handshake completion (AWREADY, WREADY) and capturing the write response (BVALID, BRESP).
- **axi_read** – Performs a complete AXI read transaction, including address phase and data phase, waiting for ARREADY and RVALID, and capturing RDATA and RRESP.

The testbench also emulates the DSP behavior by driving:

- *dsp_data_in_ready* to acknowledge input samples
- *dsp_data_out_valid* and *dsp_data_out* to provide processing results

All outputs from the DUT are monitored and validated using assertion-based verification, ensuring compliance with both functional requirements and AXI protocol behavior.

Test Scenarios

A comprehensive set of integration tests has been defined to validate both the internal functionality of the system and its compliance with the AXI-Lite protocol.

The following test scenarios are executed:

1. **Reset State:** Verifies that all AXI output signals and DSP control signals are initialized to safe default values during reset.

2. **AXI Write to CTRL and Readback:** Checks that writing configuration values through AXI correctly updates the DSP control signals and that the values can be read back through the AXI interface.
3. **Reserved Bits Protection:** Ensures that reserved bits in the CTRL register are always forced to zero, even when written with all ones through AXI.
4. **CTRL to DSP Propagation:** Verifies that writing the enable bit in the CTRL register correctly propagates to the dsp_enable signal.
5. **STATUS Register Read-Only Behavior:** Confirms that write attempts to the STATUS register are ignored while still returning a valid AXI response.
6. **Coefficient Data Write Path:** Validates that writing to the COEFF_DATA register propagates correctly to the DSP coefficient interface and generates a one-cycle dsp_coeff_we pulse.
7. **Coefficient Address Write Path:** Checks that writing to COEFF_ADDR updates the DSP coefficient address and also generates the appropriate write enable pulse.
8. **DATA_IN Write and DSP Handshake:** Verifies that writing to DATA_IN triggers the correct DSP input handshake, asserting dsp_data_in_valid and holding it until dsp_data_in_ready is received.
9. **DSP Result Capture and AXI Readback:** Ensures that DSP output data is correctly captured when dsp_data_out_valid is asserted, that dsp_data_out_ready is pulsed for one cycle, and that the result is readable via AXI.
10. **End-to-End Data Path Validation:** Performs a full write-then-read cycle for COEFF_DATA to verify data integrity across the entire AXI-to-register path.
11. **AXI Write Response (OKAY):** Confirms that valid AXI write transactions return a correct BRESP = OKAY.
12. **AXI Write Response (SLVERR):** Verifies that writes to out-of-range addresses return BRESP = SLVERR.
13. **AXI Read Response (SLVERR):** Checks that reads from invalid addresses return RRESP = SLVERR and zero data.
14. **FSM WAIT_RESULT Behavior:** Validates that the control FSM remains in the WAIT_RESULT state with STATUS.BUSY asserted across multiple cycles until a DSP result is received, after which the status transitions to READY.

Simulation Results

The simulation results demonstrate correct operation of the Top-Level Wrapper across all tested scenarios, confirming both functional correctness and AXI protocol compliance.

All AXI transactions were successfully executed following the valid/ready handshake mechanism, and correct response codes (OKAY and SLVERR) were generated depending on the validity of the accessed address. The module correctly handled out-of-range accesses without affecting internal state.

The integration between the AXI interface, register bank, and control FSM was validated end-to-end. Configuration writes propagated correctly to the DSP control signals, and data paths from AXI to DSP and back were verified to operate as expected.

The DSP handshake mechanisms were correctly implemented, with dsp_data_in_valid held until dsp_data_in_ready and dsp_data_out_ready pulsed for exactly one cycle upon result capture. The FSM behavior was also validated, ensuring correct state transitions and proper status reporting through the STATUS register.

Waveform analysis confirms that all control signals, data paths, and protocol interactions behave according to specification.

```
top_level_wrapper_tb.vhd:272:9:@0ms:(report note): Test 1: Verifying AXI output reset state
top_level_wrapper_tb.vhd:312:9:@50ns:(report note): Test 2: Verifying AXI write to CTRL and readback
top_level_wrapper_tb.vhd:342:9:@230ns:(report note): Test 3: Verifying CTRL reserved bits are forced to zero
top_level_wrapper_tb.vhd:362:9:@410ns:(report note): Test 4: Verifying CTRL enable bit propagates to dsp_enable
top_level_wrapper_tb.vhd:385:9:@525ns:(report note): Test 5: Verifying STATUS is read-only (AXI write ignored)
top_level_wrapper_tb.vhd:411:9:@715ns:(report note): Test 6 START: Verifying COEFF_DATA write propagates to DSP coefficient interface
top_level_wrapper_tb.vhd:437:9:@835ns:(report note): Test 7: Verifying COEFF_ADDR write propagates to dsp_coeff_addr
top_level_wrapper_tb.vhd:460:9:@905ns:(report note): Test 8: Verifying DATA_IN write triggers DSP sample send
top_level_wrapper_tb.vhd:498:9:@1035ns:(report note): Test 9: Verifying DSP result captured and readable via AXI
top_level_wrapper_tb.vhd:539:9:@1210ns:(report note): Test 10: Verifying full write/read round trip for COEFF_DATA
top_level_wrapper_tb.vhd:557:9:@1325ns:(report note): Test 11: Verifying BRESP=OKAY for valid AXI write
top_level_wrapper_tb.vhd:571:9:@1385ns:(report note): Test 12: Verifying BRESP=SLVERR for out-of-range AXI write
top_level_wrapper_tb.vhd:585:9:@1445ns:(report note): Test 13: Verifying RRESP=SLVERR for out-of-range AXI read
top_level_wrapper_tb.vhd:604:9:@1515ns:(report note): Test 14: Verifying STATUS.BUSY held in WAIT_RESULT until result arrives
top_level_wrapper_tb.vhd:651:9:@2115ns:(report note): End of simulation
```

Figure 27– Test results

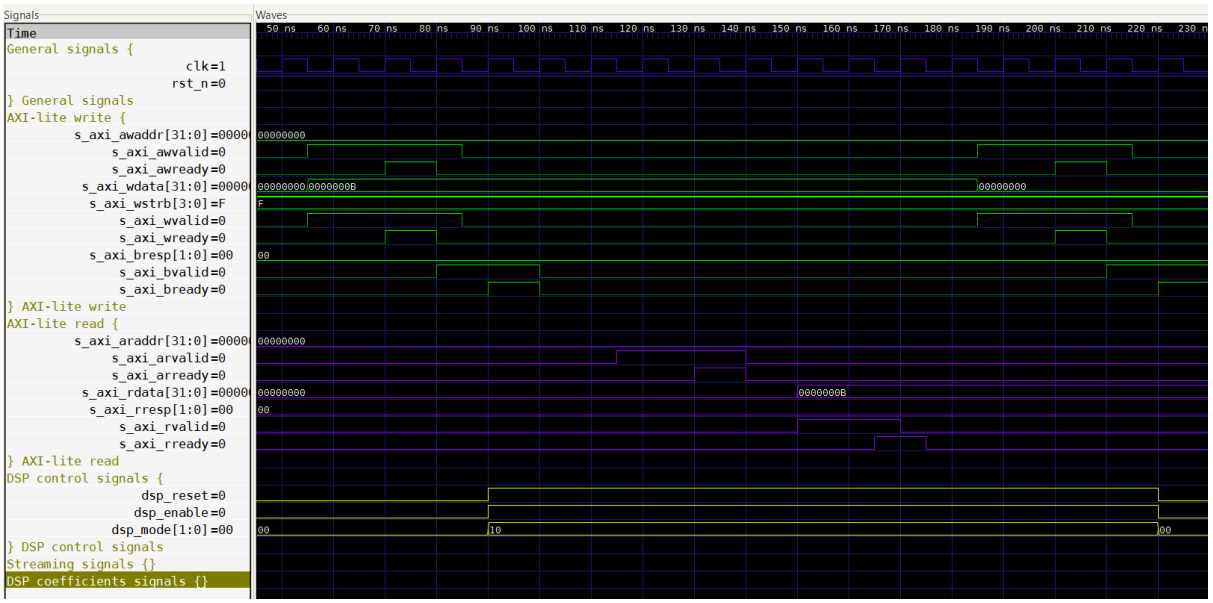

Figure 28 – Test 2

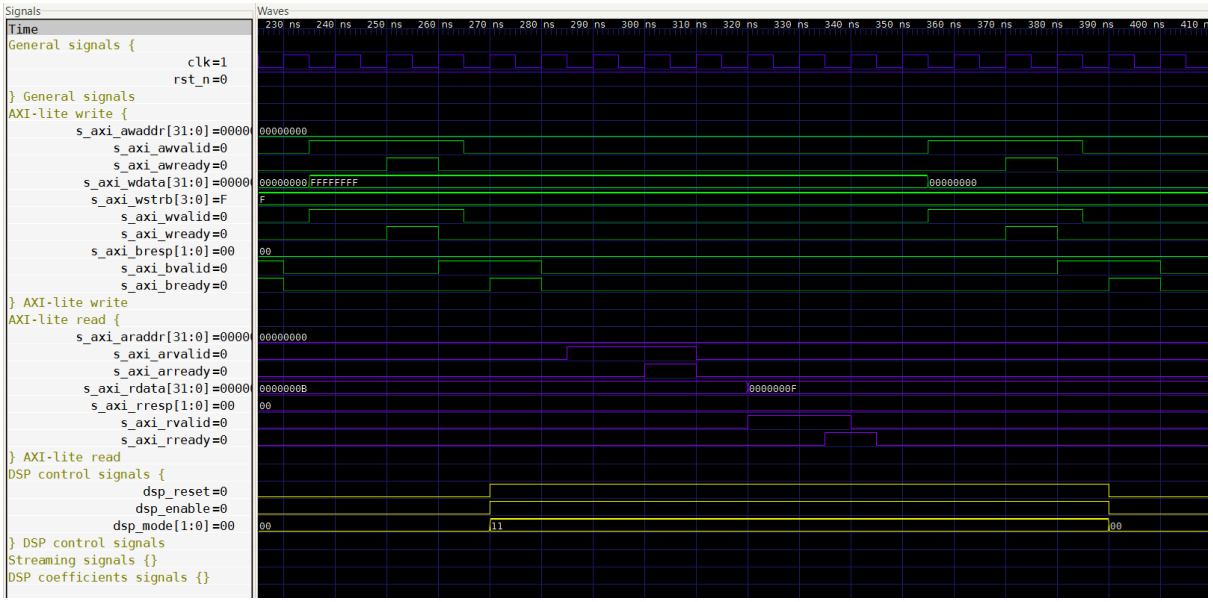

Figure 29 – Test 3

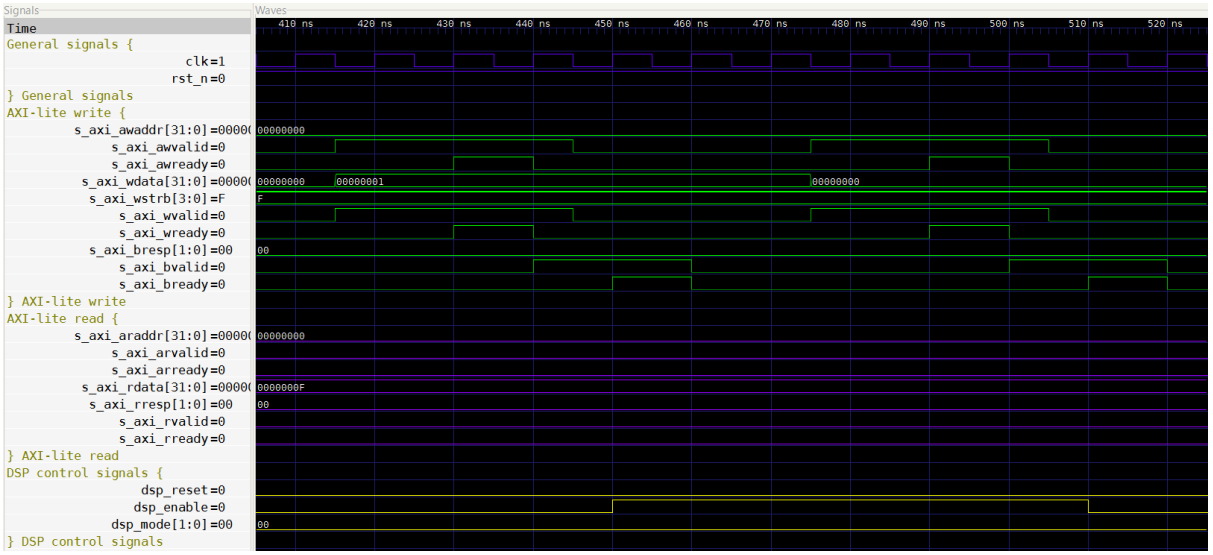

Figure 30 – Test 4

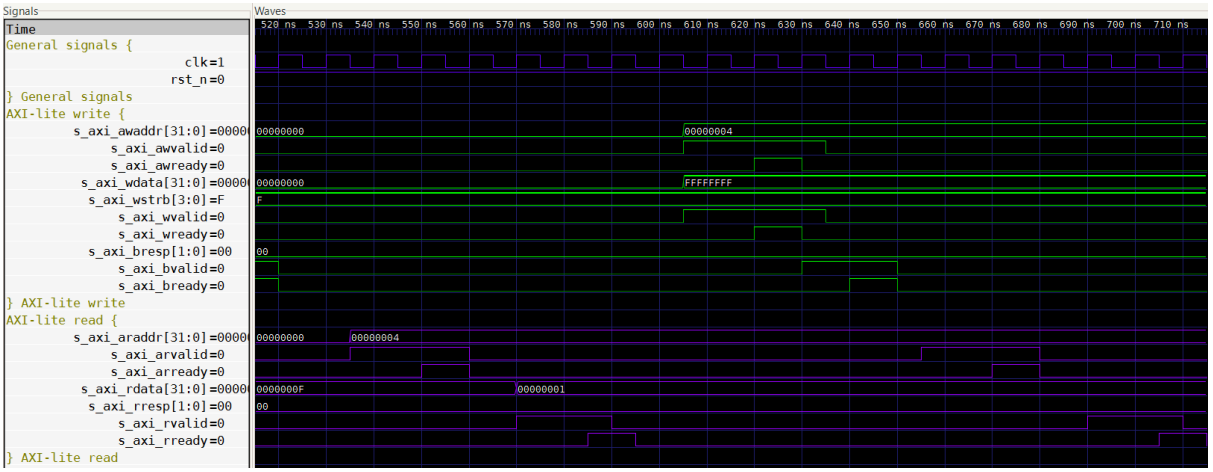

Figure 31 – Test 5

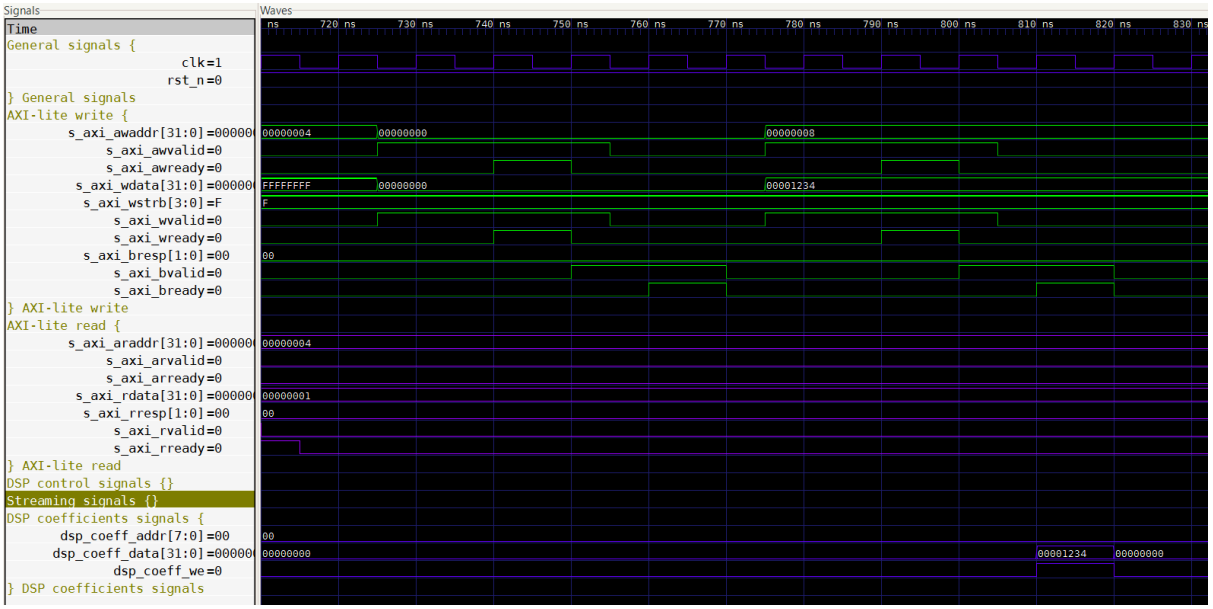

Figure 32 – Test 6

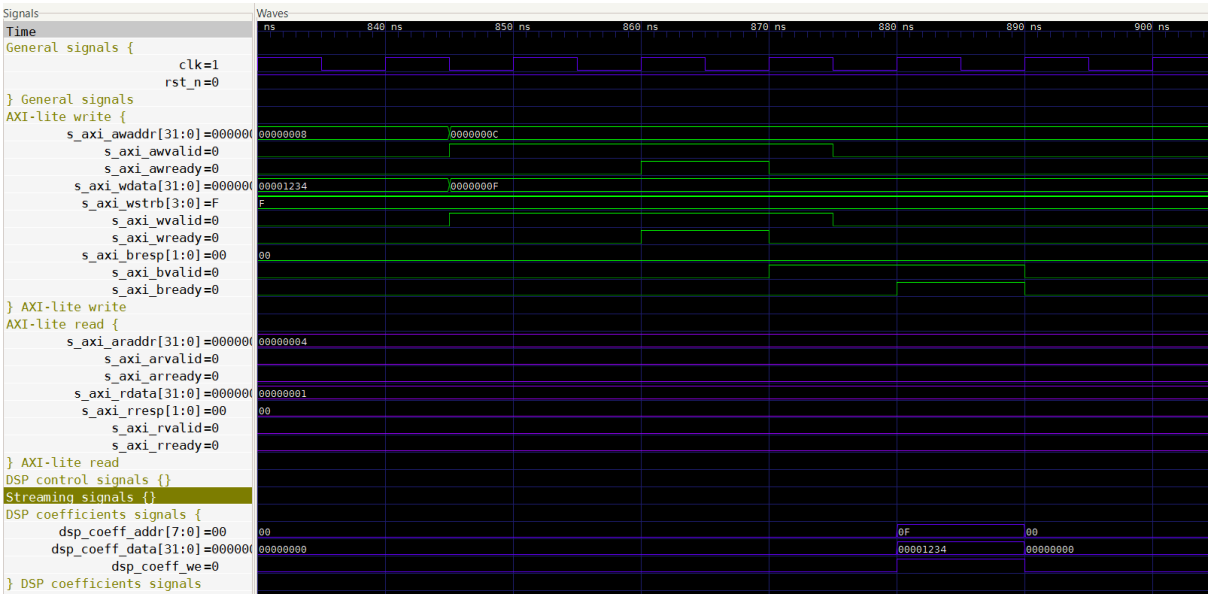

Figure 33 – Test 7

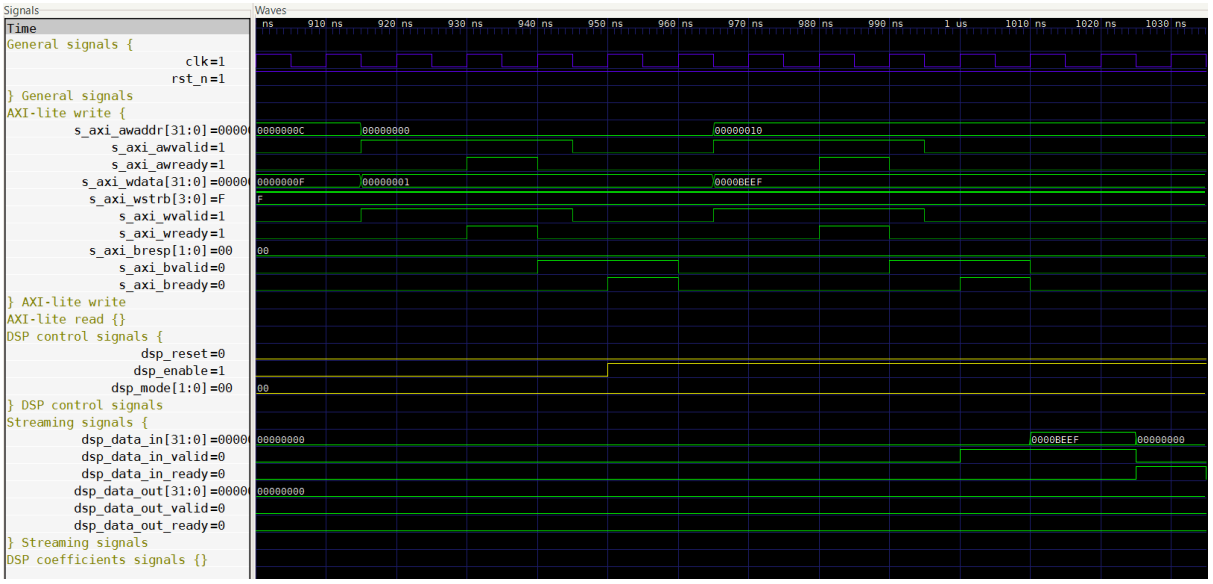

Figure 34 – Test 8

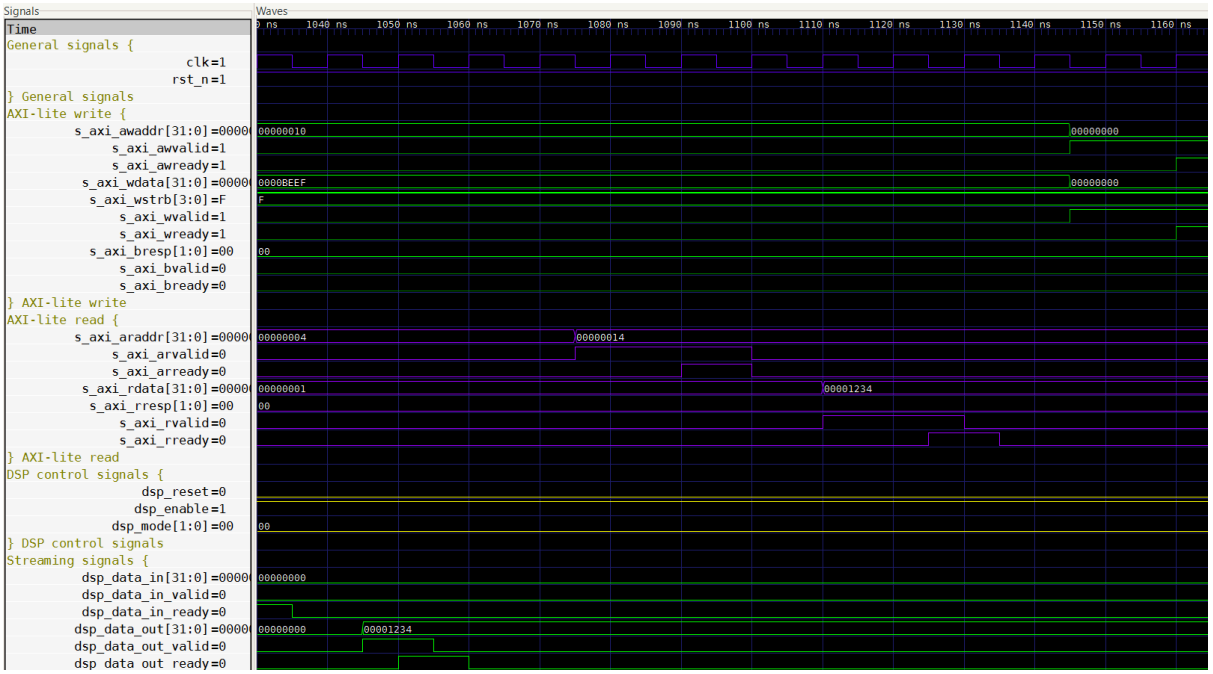

Figure 35 – Test 9

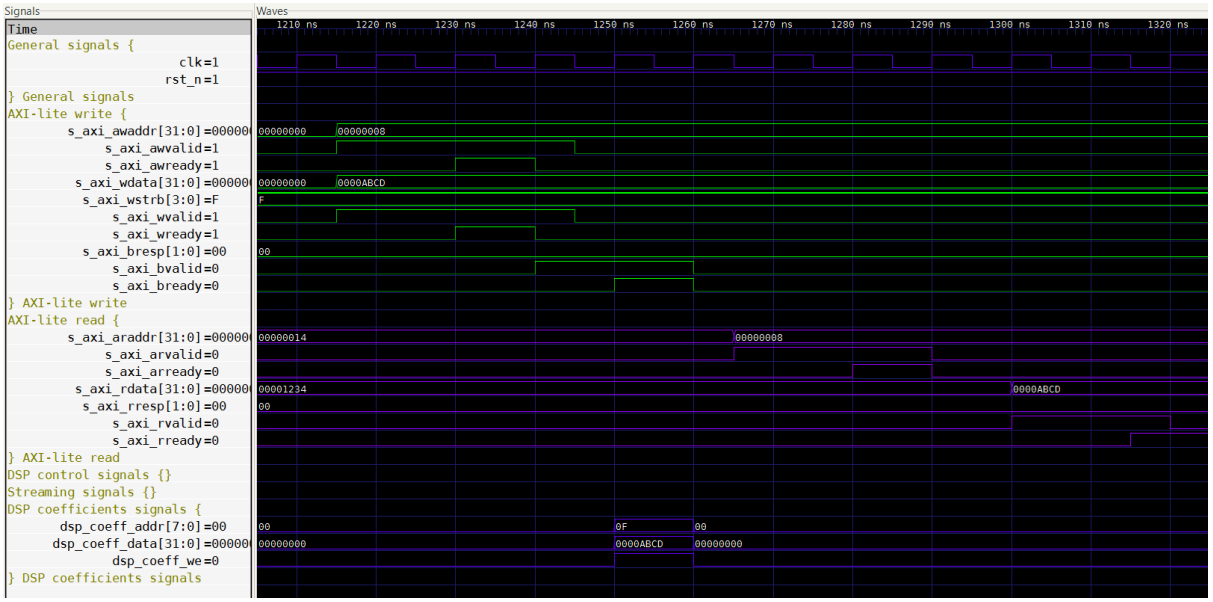

Figure 36 – Test 10

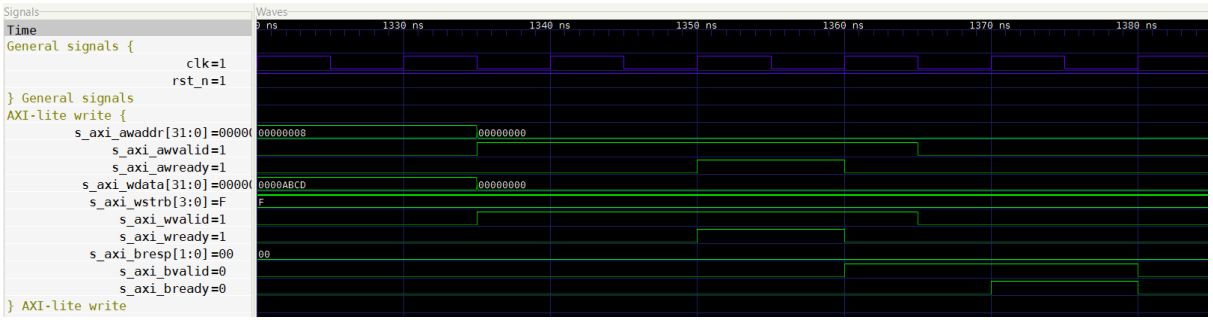

Figure 37 – Test 11

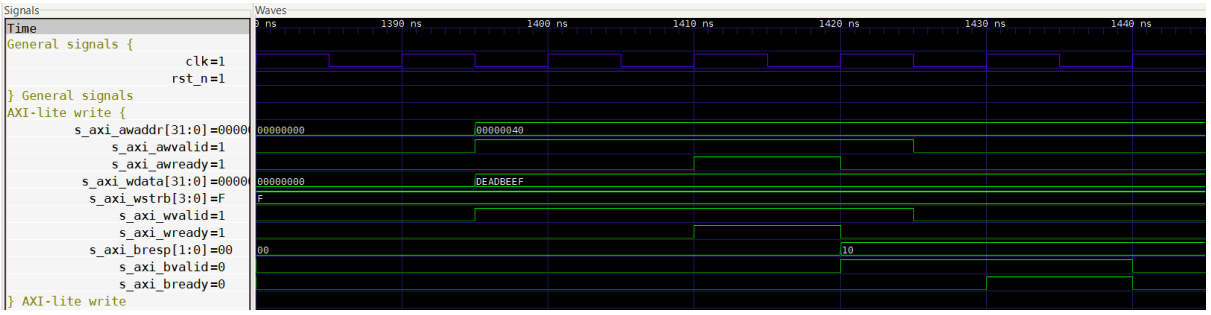

Figure 38 – Test 12

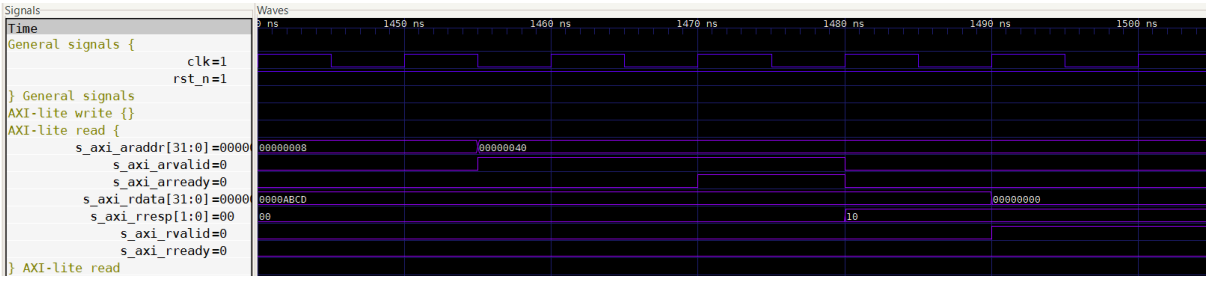

Figure 39 – Test 13

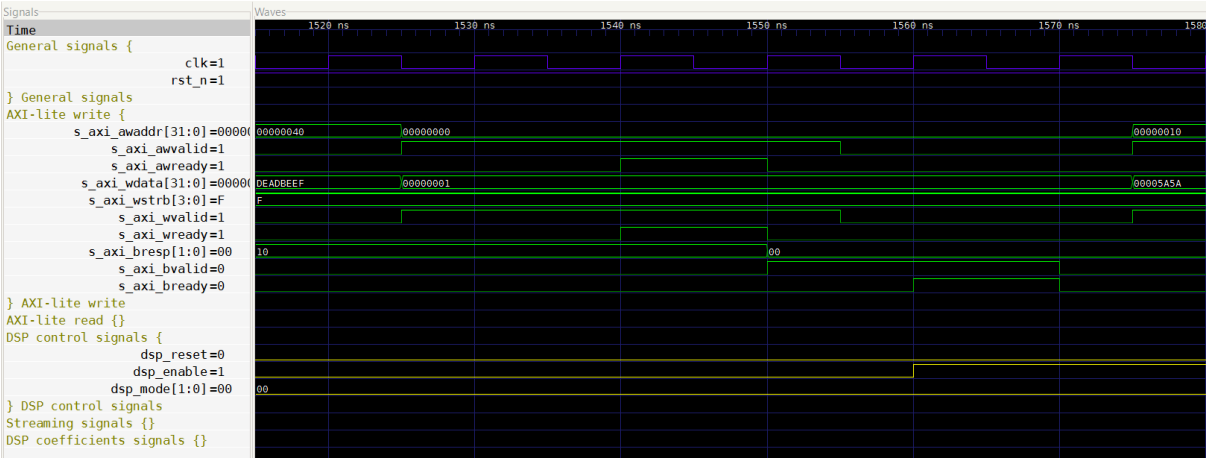

Figure 40 – Test 14